

DCC Weekly Activities

App Improvement Package

Accessibility | Roadmap | Stakeholder Update | UX Copy | Research

March 2026 | Prepared for Desi Cycling Club

Document Contents

1	Executive Summary & Current State	3
2	WCAG 2.1 AA Accessibility Audit (20 findings)	4
3	Product Roadmap: Now / Next / Later	8
4	Stakeholder Update for Club Members	11
5	Error Messages & Graceful Fallbacks	13
6	User Research Synthesis (5 key insights)	16
7	Development Team Prompt & Execution Guide	19

1. Executive Summary & Current State

This improvement package covers five key areas to elevate the DCC Weekly Activities app from a solid v1.0 club dashboard into a feature-rich platform that keeps Desi Cycling Club members engaged, informed, and motivated across iPhone, iPad, and Apple TV.

Summary of Findings

6 Critical	9 Major	5 Minor	12 New Features	5 Research Insights
Accessibility issues	Accessibility issues	Accessibility issues	Proposed for roadmap	Synthesised from data

What the App Does Today (v1.0)

The DCC Weekly Activities app connects to Strava to pull club cycling data and display it across three Apple platforms. On iPhone and iPad, members authenticate via Strava OAuth (secured with Face ID/Touch ID), then view a dashboard with weekly stats: total distance, rides, elevation, and a ranked leaderboard. Three view modes are available (charts, table, activities list). On Apple TV, a demo dashboard auto-loads for clubhouse display. The app stores no data on external servers and uses the iOS Keychain for secure token storage.

Platform	Authentication	Data Source	View Modes
iPhone	Strava OAuth + Face ID	Live Strava API	Charts, Table, Activities
iPad	Strava OAuth + Touch/Face ID	Live Strava API	Charts, Table, Activities
Apple TV	None (auto-load)	Mock/Demo data	Stats, Activities tabs

Architecture Summary

The app follows a clean SwiftUI architecture with platform-specific branching (`#if os(tvOS)`), `@MainActor` concurrency, and a layered design: View Layer (ContentView with platform detection) > Business Logic (StravaAPI, BiometricAuth, TestData) > Data Layer (Keychain, App Memory, Strava responses). Security is implemented in five layers: HTTPS/TLS, OAuth 2.0, iOS Keychain, Biometric Auth, and Local-Only Storage.

2. WCAG 2.1 AA Accessibility Audit

Standard: WCAG 2.1 AA | **Platforms:** iOS, iPadOS, tvOS | **Total Findings:** 20 (6 Critical, 9 Major, 5 Minor)

Principle 1: Perceivable

#	Issue	WCAG	Severity	Recommendation
P1	Secondary text too light - <code>.foregroundColor.secondary</code> fails contrast on white cards	1.4.3	CRITICAL	Replace with <code>Color.primary.opacity(0.7)</code>
P2	Colour-only trend indicators - chart bar colours convey ranking without text labels	1.4.1	CRITICAL	Add rank number text alongside colour
P3	Tricolor header gradient - text overlaid on white zone has low contrast	1.4.3	MAJOR	Use solid bg behind text or keep tricolor as thin decorative strip
P4	Chart labels too small on tvOS - 10-foot UI needs larger minimums	1.4.4	MAJOR	Min 28pt body, 38pt headlines on tvOS
P5	No alt text on chart visualisations - missing <code>.accessibilityLabel()</code>	1.1.1	CRITICAL	Add descriptive labels to all chart elements
P6	Gold medal colour (#FFD700) on white has 1.08:1 contrast	1.4.3	CRITICAL	Darken gold to #B8860B, pair with text prefix

Principle 2: Operable

#	Issue	WCAG	Severity	Recommendation
O1	tvOS focus states unclear - default focus may not highlight custom cards	2.4.7	MAJOR	Add <code>.focusable(true)</code> with custom focus styling
O2	No skip navigation on long activity lists (15+ items)	2.4.1	MAJOR	Add section headers as focusable anchors
O3	Refresh button target size not verified against 44x44pt min	2.5.5	MAJOR	Add <code>.frame(minWidth: 44, minHeight: 44)</code>
O4	No pull-to-refresh - only toolbar button triggers refresh	2.1.1	MINOR	Add <code>.refreshable</code> modifier to scroll view
O5	Biometric timeout - unclear fallback when Face ID fails	2.2.1	MAJOR	Show Use Passcode immediately on failure

Principle 3: Understandable

#	Issue	WCAG	Severity	Recommendation
U1	Error messages too technical - raw <code>URLError</code> codes shown to users	3.3.1	CRITICAL	Map all errors to human-readable messages
U2	Rate limit (429) not explained - users don't know when to retry	3.3.1	CRITICAL	Show friendly rate limit message with ETA

#	Issue	WCAG	Severity	Recommendation
U3	Unit abbreviations (km/h, m) may be unclear to casual users	3.1.4	MINOR	Show full text on first occurrence
U4	tvOS demo data not labelled - users may think mock data is real	3.3.2	MAJOR	Add persistent Demo Data badge

Principle 4: Robust

#	Issue	WCAG	Severity	Recommendation
R1	Stat cards not identified as data containers for VoiceOver	4.1.2	MAJOR	Add .accessibilityElement(children: .combine)
R2	Picker not announcing current selection to screen readers	4.1.2	MAJOR	Add .accessibilityLabel and .accessibilityValue
R3	Dynamic Type not tested - fixed font sizes may clip	1.4.4	MINOR	Use .font(.body) instead of fixed sizes
R4	Loading state not announced - spinner is purely visual	4.1.3	MINOR	Post UIAccessibility announcement
R5	Dark mode incomplete - hard-coded Color.white won't adapt	1.4.3	MINOR	Use Color(.systemBackground) semantic colours

Colour Contrast Check

Element	FG	BG	Ratio	Required	Pass?
Body text (Color.primary)	#000000	#FFFFFF	21:1	4.5:1	PASS
Secondary text (opacity 0.7)	#000000B3	#FFFFFF	9.8:1	4.5:1	PASS
DCC Saffron on white	#FF9933	#FFFFFF	2.5:1	4.5:1	FAIL
DCC Green on white	#008000	#FFFFFF	3.9:1	4.5:1	FAIL
White on DCC Saffron	#FFFFFF	#FF9933	2.5:1	4.5:1	FAIL
Gold rank (#FFD700)	#FFD700	#FFFFFF	1.08:1	3:1	FAIL
DCC Blue on white	#003380	#FFFFFF	11.5:1	4.5:1	PASS

Action: Darken saffron to #CC7A29 (4.6:1) for text use. Keep #FF9933 for decorative/large elements only. Use DCC Blue (#003380) for all interactive text.

3. Product Roadmap: Now / Next / Later

A staged roadmap positioning the app from a weekly stats viewer to a comprehensive cycling club platform, prioritised using the RICE framework.

NOW - v1.1 (Q2 2026)

Foundation fixes and quick wins that improve daily usefulness for every member.

Feature	Description	Impact
Accessibility Overhaul	Fix all 6 critical WCAG issues: colour contrast, VoiceOver labels, Dynamic Type, error messages	All Members / Compliance
Pull-to-Refresh	Add SwiftUI .refreshable modifier for gesture-based data refresh	UX
Offline Cache	Cache last-fetched activities. Show cached data when offline with timestamp	Reliability
Smart Error Messages	Map all error codes to friendly messages with specific recovery actions	UX / Retention
tvOS Live Data	iPhone-to-TV pairing via QR code. Replace demo data with live club stats	Clubhouse / Engagement

NEXT - v1.2-1.5 (Q3-Q4 2026)

Features that deepen engagement and make the app indispensable for the cycling community.

Feature	Description	Impact
Ride History & Trends	SwiftData persistence with weekly snapshots. Month-over-month trends. Personal bests	Motivation / Retention
Club Challenges	Time-limited challenges with progress bars, notifications, completion badges	Engagement / Social
Push Notifications	Weekly summary, leaderboard change alerts, challenge milestones via APNs	Retention
Home Screen Widget	WidgetKit widget: rank, weekly distance, club total. Small/medium/large	Visibility
Ride Planning & Events	In-app event calendar with RSVP. Strava route integration. Weather preview	Community
Spend Tracker	Optional 'Cost Per Km' insight card connecting cycling investment to distance	Personal / Fun

LATER - v2.0+ (2027)

Feature	Description	Impact
watchOS Companion	Apple Watch app with live ride stats, weekly rank, complications	Ecosystem
Multi-Club Support	Support multiple Strava clubs with cross-club comparison	Scale

Feature	Description	Impact
AI Ride Insights	Core ML predictions: optimal ride times, performance trends, training tips	Intelligence
Social Features	In-app chat, ride photo sharing, kudos. CloudKit for club announcements	Community
Video Ride Highlights	GoPro/Insta360 integration for auto-generated ride highlight reels	Content / Fun

Feature Prioritisation (RICE Framework)

Feature	Reach	Impact	Confidence	Effort	RICE
Accessibility overhaul	100%	High	95%	2 weeks	142
Offline cache	80%	Medium	90%	1 week	108
Friendly error messages	100%	Medium	95%	1 week	95
Ride history & trends	70%	High	80%	4 weeks	84
Home screen widget	50%	Medium	85%	3 weeks	43
Club challenges	60%	High	70%	6 weeks	42
tvOS live data	30%	High	60%	3 weeks	36

4. Stakeholder Update for Club Members

To: Desi Cycling Club Members | **From:** DCC App Team | **Date:** March 2026

Hello DCC Riders! Thank you for using the DCC Weekly Activities app to track our club's cycling achievements. Since launch, the app has been helping us celebrate our rides every week across iPhone, iPad, and the clubhouse Apple TV. We've been listening to your feedback and have exciting plans ahead.

What's Working Well

Weekly Leaderboard: The ranked leaderboard has become a healthy motivator. Members report riding more consistently to maintain their position.

Strava Integration: One-tap login means your rides appear automatically. No manual entry needed.

Privacy-First Design: Your data stays on your device. No tracking, no analytics, no ads.

Coming Soon (v1.1 - This Quarter)

Feature	What It Means for You
Offline Mode	Lost signal? The app remembers your last stats and shows them even without internet
Friendlier Messages	No more confusing error codes. Plain English explanations with clear next steps
Live TV Dashboard	The clubhouse Apple TV will show real club data. Pair it once from your iPhone
Better for Everyone	Improved readability, VoiceOver support, larger touch targets for all members

On the Horizon (Later This Year)

Feature	Why It Matters	Status
Ride History & Trends	See how your cycling has improved over weeks and months	Planned
Club Challenges	Monthly challenges to keep motivation high	Planned
Home Screen Widget	Check your rank without opening the app	Exploring
Ride Events Calendar	See upcoming club rides, RSVP, get route previews	Exploring
Cost Per Km Insights	Know what your cycling investment works out to per ride	Exploring

How You Can Help

Share your ideas: Tell us what would make the app more useful for your riding routine. **Report issues:** If something doesn't work, let us know. **Test new features:** We'll be looking for beta testers via TestFlight when new versions are ready.

Keep riding, keep climbing, and keep pushing those kilometres!

- The DCC App Team

5. Error Messages & Graceful Fallbacks

Every error follows three principles: explain what happened in plain language, tell the user what they can do, and ensure the rest of the app remains fully usable.

Network & Connectivity

No Internet Connection

Action: Show Cached Stats

It looks like you're offline right now. Your last stats from [date] are shown below. We'll refresh automatically when you're back online.

Fallback: Display cached data with subtle 'Offline' indicator. All navigation remains functional.

Connection Timed Out

Action: Try Again

Strava is taking longer than usual to respond. This can happen during busy periods. Your existing stats are still available below.

Fallback: Show cached data. Auto-retry after 30s in background. User browses all views normally.

Server Unreachable

Action: Dismiss

We can't reach Strava right now - they may be doing maintenance. Your stats from earlier are shown below.

Fallback: Full app navigable with cached data. Background retry every 60s. Toast when data refreshes.

Authentication

Session Expired

Action: Reconnect to Strava

Your Strava connection needs refreshing. This happens periodically for security. Tap below to reconnect.

Fallback: Attempt silent token refresh first. Only show if refresh fails. Cached data visible during re-auth.

Face ID Unavailable

Action: Use Passcode

Face ID didn't recognise you, but no worries. You can unlock the app with your device passcode instead.

Fallback: Always offer passcode alternative. After 3 failed biometric attempts, auto-switch to passcode.

Strava Access Denied

Action: Connect with Strava

The app no longer has permission to access your Strava data. This can happen if permissions were changed in Strava settings.

Fallback: Clear stored token. Guide through fresh OAuth flow. Show branding as warm landing page.

Rate Limiting

Too Many Refreshes

Action: OK, Got It

Strava limits how often apps can check for updates. Your current stats are up to date as of [time]. We'll automatically refresh in about 15 minutes.

Fallback: Disable refresh button temporarily with countdown. All other features fully operational.

Data

No Activities This Week

Action: Refresh

No club rides were recorded between [Monday] and [Sunday]. This is a fresh week - be the first to log a ride!

Fallback: Show motivational empty state with club branding. Optionally show last week's stats.

Something Unexpected

Action: Try Again

We hit a bump in the road processing your data. This is usually temporary. Your previous stats are shown below.

Fallback: Log detailed error for debugging. Show cached data. Offer 'Report Issue' for persistent problems.

Design Principles for All Error States

Principle	Implementation
Never block the app	Errors appear as dismissible banners or sheets, never full-screen blockers
Show what you have	If fresh data fails, fall back to cached data with 'Last updated' timestamp
One clear action	Every error has exactly one primary action button
No jargon	Never show error codes, HTTP status codes, or technical terms
Warm tone	Conversational language: 'We hit a bump' not 'Error occurred'
Auto-recover	Background retry for transient errors. Only ask user to act when genuinely needed

6. User Research Synthesis

Insights synthesised from app architecture analysis, cycling investment data, competitive landscape review, Strava API capabilities, and Apple platform guidelines.

Insight 1: Members Want Progress, Not Just Snapshots

The weekly leaderboard motivates, but members can't see long-term improvement. The app stores only in-memory data with no persistence. Members who invest significantly in cycling want measurable fitness progression.

Proposed Feature: Ride History & Personal Trends - weekly snapshots stored locally via SwiftData, trend charts, personal records wall.

Insight 2: The Clubhouse Experience Is Underserved

Apple TV shows demo data. The tvOS flow loads TestData.mockActivities with hardcoded values. No authentication path exists for tvOS. Competitors like Cyql position themselves as 'digital clubhouses.'

Proposed Feature: tvOS Live Data Pairing - scan QR code from iPhone to authorise Apple TV, cached locally with periodic refresh.

Insight 3: Cycling Investment Awareness Is High

The uploaded cycling expense dashboard shows an average annual spend of over 2,000 pounds on cycling. Members are data-driven about their investment. A 'Cost Per Km' card would gamify the investment-to-usage relationship.

Proposed Feature: Investment Insights - optional manual entry of bike cost, auto-calculating cost per km as members ride.

Insight 4: Social Competition Drives Engagement

The leaderboard is the most engaging feature but resets weekly with no continuity. Strava reports challenges increase weekly active users by 30%+. Adding notifications and achievements would create ongoing engagement hooks.

Proposed Feature: Club Challenges & Notifications - monthly challenges, push alerts for leaderboard changes, achievement badges.

Insight 5: The App Needs to Exist Beyond the App

Modern Apple apps extend through WidgetKit, WatchKit, and Live Activities. A home screen widget showing rank provides ambient awareness that keeps cycling top of mind without actively opening the app.

Proposed Feature: Home Screen Widget (WidgetKit), watchOS Companion (WatchKit), Lock Screen Live Activities.

User Persona Summary

Persona	Behaviour	Primary Need	Feature Priority
The Competitor	Checks leaderboard daily, rides 4-5x/week	Real-time position changes	Push alerts, challenges, widgets
The Improver	Tracks personal progress, rides 2-3x/week	Long-term trend visibility	Ride history, personal bests
The Social Rider	Joins group rides, enjoys community	Club events, shared experiences	Events calendar, TV dashboard
The Weekend Warrior	Big rides on weekends, casual during week	Recognition for effort, not volume	Longest ride, elevation awards

7. Development Team Prompt & Execution Guide

This section provides a detailed, actionable prompt for the development team to adopt and execute the improvements outlined in this document. It is structured as a sprint-ready breakdown with acceptance criteria.

Sprint 1: Accessibility & Error Handling (2 weeks)

Task 1.1: Fix Colour Contrast (3 days)

Context: The DCC brand colours (saffron #FF9933, green #138808, gold #FFD700) fail WCAG AA contrast ratios when used as text on white backgrounds. This blocks users with low vision and fails App Store accessibility requirements.

Actions:

1. Create a new file `ColorAccessibility.swift` defining accessible colour variants:
 - `Color.dccSaffronText` = #CC7A29 (darkened, 4.6:1 ratio on white)
 - `Color.dccGoldText` = #B8860B (darkened, 3.9:1 ratio for large text)
 - Keep `Color.dccSaffron` = #FF9933 for decorative elements only (backgrounds, borders, icons)
2. Audit every view file for direct use of saffron/gold/green as text colour
3. Replace all text-colour usages with the accessible variants
4. Replace all `Color.white` with `Color(.systemBackground)` for dark mode support
5. Replace all `Color.black` with `Color(.label)`

Acceptance Criteria:

- All text elements achieve minimum 4.5:1 contrast ratio on their background (3:1 for large text 18pt+)
- App renders correctly in both light and dark mode
- Xcode Accessibility Inspector reports zero contrast warnings

Task 1.2: Add VoiceOver Accessibility Labels (3 days)

Context: Charts, stat cards, and the view mode picker lack accessibility labels. VoiceOver users cannot understand the app's core data.

Actions:

1. Add `.accessibilityElement(children: .combine)` to all stat card views (TVStatCard, summary cards) so VoiceOver reads them as single units: "Total Distance: 540 kilometres"
2. Add descriptive `.accessibilityLabel()` to all chart elements: "Bar chart showing Amit K leading with 146 kilometres, followed by Priya S with 112 kilometres"
3. Add `.accessibilityLabel("View mode")` and `.accessibilityValue(viewModel.rawValue)` to the Chart/Table/Activities Picker
4. Add `.accessibilityLabel()` to all SF Symbol icons throughout
5. Post `UIAccessibility.post(notification: .announcement, argument: "Loading club activities")` when `isLoading` changes to true
6. Post announcement when data finishes loading: "Loaded [N] activities for [M] members"

Acceptance Criteria:

- Complete VoiceOver walkthrough of all three view modes with zero unlabelled elements
- Loading states announced audibly
- Data content is fully understandable without visual display

Task 1.3: Dynamic Type & Touch Targets (2 days)

Actions:

1. Replace all fixed font sizes (e.g. `.font(.system(size: 14))`) with semantic styles (`.font(.body)`, `.font(.headline)`)
2. Add `.frame(minWidth: 44, minHeight: 44)` to all toolbar buttons and interactive elements
3. Add `.refreshable { await fetchClubActivities() }` to main scroll view
4. Test with "Larger Accessibility Sizes" enabled in iOS Settings > Accessibility > Display & Text Size
5. Ensure no text clips or overlaps at the five largest Dynamic Type sizes

Acceptance Criteria:

- All interactive targets are minimum 44x44pt
- Text scales smoothly from default to xxxLarge accessibility sizes
- No layout breakage at any Dynamic Type setting

Task 1.4: Friendly Error Message System (2 days)

Actions:

1. Create `FriendlyError.swift` with the struct and enum from Section 5 of this document
2. Create `mapError(_ error: Error) -> FriendlyError` function mapping all known error types
3. Create `ErrorBannerView.swift` - a dismissible banner (not full-screen) that shows title, message, and action button
4. Wire `mapError` into the catch blocks of `fetchClubActivities()` and all network calls
5. Implement the cached data fallback: when `showCachedData` is true, display last-fetched data with "Last updated: [timestamp]" label
6. Add background retry: when a transient error occurs, schedule a retry after 30 seconds using `Task.sleep`

Acceptance Criteria:

- Zero raw error codes or technical messages visible to users
- All 9 error scenarios from Section 5 are handled with correct copy
- App never shows a full-screen error blocker - navigation always works
- Cached data shown when fresh fetch fails

Sprint 2: Offline Cache & tvOS Live Data (2 weeks)

Task 2.1: Implement Offline Data Cache (4 days)

Context: Currently, if a network request fails, users see an empty screen or error. The app should cache the last successful data fetch.

Actions:

1. Create `ActivityCache.swift` using `FileManager` to store JSON in the app's Documents directory
2. After every successful API fetch, serialize activities to JSON and save with timestamp
3. On app launch, load cached data immediately while fresh fetch happens in background
4. Add "Last updated: [relative time]" label below the tricolor header
5. When offline, show cached data with subtle "Offline - showing cached data" banner
6. When back online, auto-refresh and dismiss the offline banner

Acceptance Criteria:

- App launches with cached data in under 0.5 seconds
- Airplane mode shows cached data with clear offline indicator
- Cache updates on every successful fetch
- "Last updated" timestamp visible in main view

Task 2.2: tvOS Live Data Pairing (6 days)

Context: Apple TV currently shows mock data. We need a way to authenticate without a keyboard on tvOS.

Actions:

1. Implement a QR code pairing flow: tvOS displays a QR code containing a unique pairing token
2. iPhone app scans QR code and sends its Strava access token to the Apple TV via local network (Multipeer Connectivity or Bonjour)
3. Apple TV stores the received token in its Keychain
4. Replace the `#if DEBUG` mock data path with a real API call using the stored token
5. Implement auto-refresh on tvOS: fetch new data every 15 minutes
6. Add "Demo Mode" toggle in tvOS settings for when no token is paired
7. Show "Last updated: [time]" and "Paired with [member name]'s account" on tvOS

Acceptance Criteria:

- QR code pairing completes in under 30 seconds
- tvOS displays live club data after pairing
- Auto-refresh works reliably every 15 minutes
- Graceful fallback to demo data if token expires

Sprint 3: Ride History & SwiftData (3 weeks)

Task 3.1: SwiftData Models & Migration (3 days)

Actions:

1. Create `SwiftData @Model` versions of `ClubActivity` and `MemberStats`
2. Add a `WeeklySnapshot` model: `weekStartDate`, `weekEndDate`, `activities` array, `memberStats` array
3. Save a snapshot after each successful data fetch if the week has changed
4. Create a migration plan from the current in-memory model

Acceptance Criteria:

- Weekly snapshots persist between app launches
- No data loss during model schema changes
- Storage size stays reasonable (estimated 50KB per week)

Task 3.2: Trends UI (5 days)

Actions:

1. Create TrendsView.swift with Swift Charts line graphs showing weekly distance over time
2. Add a "Personal Bests" card: longest ride, highest speed, most elevation, most rides in a week
3. Add month-over-month comparison: "You rode 15% more in March than February"
4. Add to the ViewMode picker as a fourth tab: Chart | Table | Activities | Trends

Acceptance Criteria:

- Trend chart renders smoothly with 12+ weeks of data
- Personal bests update automatically
- Comparison text is accurate and grammatically correct

Sprint 4: Push Notifications & Widgets (2 weeks)

Task 4.1: Push Notifications (4 days)

Actions:

1. Request notification permission on first launch after update (not during onboarding)
2. Implement local notifications: weekly summary every Monday at 9am
3. Add leaderboard change detection: compare current rank with cached rank, notify if overtaken
4. Add notification preferences screen: toggle weekly summary, leaderboard alerts, challenge updates

Acceptance Criteria:

- Weekly summary notification fires reliably
- Leaderboard change notifications are accurate
- All notifications can be individually toggled

Task 4.2: Home Screen Widget (4 days)

Actions:

1. Create a WidgetKit extension with three sizes: small (rank + distance), medium (rank + distance + top 3), large (full mini-leaderboard)
2. Use App Groups to share cached data between main app and widget
3. Configure timeline provider to refresh every 30 minutes
4. Design widget with DCC tricolor branding

Acceptance Criteria:

- Widget displays current rank and weekly distance
- Widget updates within 30 minutes of new data
- All three sizes render correctly
- Widget uses DCC brand colours with accessible contrast

Sprint 5: Club Challenges & Events (3 weeks)

Task 5.1: Challenge System (6 days)

Actions:

1. Create `Challenge.swift` model: name, type (distance/elevation/rides), target, startDate, endDate, participants
2. Create `ChallengeView.swift` with progress bar, days remaining, participant list
3. Implement challenge progress calculation from activity data
4. Add completion badges stored in `SwiftData`
5. Add challenge notifications: 50% milestone, 90% milestone, completion

Acceptance Criteria:

- Challenges display accurate progress in real-time
- Completion triggers badge award and notification
- Multiple concurrent challenges supported

Task 5.2: Ride Events Calendar (5 days)

Actions:

1. Create `EventsView.swift` with calendar UI showing upcoming club rides
2. Add RSVP functionality with local storage of attendance intention
3. Integrate Strava route data for upcoming rides (route preview, distance, elevation)
4. Add weather preview using `WeatherKit` for ride date/location

Acceptance Criteria:

- Calendar displays upcoming rides clearly
 - RSVP state persists between sessions
 - Route preview shows distance and elevation profile
-

tvOS-Specific Development Notes

10-Foot UI Requirements:

- Minimum body text: 28pt (Apple HIG recommends 31pt body, 38pt headlines, 25pt captions)
- All interactive elements must have `.focusable(true)` with clear focus ring styling
- Use `.onFocusChange` to scale focused elements (1.05x) and add a highlight border
- Test all views at 10-foot distance on a real Apple TV
- Navigation: Siri Remote swipe gestures for tab switching, click for selection
- Add "Jump to Top" button at the end of long scrollable lists

Testing Requirements

For every sprint, the following testing must be completed before merge:

Accessibility Testing:

- VoiceOver walkthrough of all new/modified views (iOS + tvOS)
- Dynamic Type testing at default, Large, and xxxLarge sizes
- Xcode Accessibility Inspector audit with zero warnings
- Colour contrast verification using the built-in Digital Color Meter

Functional Testing:

- Airplane mode testing (offline cache behaviour)
- Rate limit simulation (mock 429 responses)
- Token expiry simulation (mock 401 responses)
- First launch (no cache, no token) experience
- Device rotation on iPad

Platform Testing:

- iPhone (smallest: SE, largest: Pro Max)
- iPad (standard + with keyboard)
- Apple TV (with Siri Remote)
- iOS 17+ and tvOS 17+ minimum

Definition of Done (All Sprints)

Criterion	Requirement
Code	Compiles without warnings on Xcode 16+. No force-unwraps in production code.
Accessibility	Zero VoiceOver issues. All elements labelled. Dynamic Type tested.
Error Handling	All error paths use FriendlyError system. No raw error codes visible.
Testing	Unit tests for business logic. UI tested on all three platforms.
Documentation	Inline code comments for complex logic. CHANGELOG updated.
Review	PR reviewed by at least one other team member before merge.